Chapter 62: Role-Based Access Control

Scenario 1: Deciding where "authorization" belongs in a standalone app

Situation: A team building a new standalone Angular app (no NgModules) asks you to design the RBAC layer from scratch. They have roles like `admin`, `manager`, `viewer`, and a growing list of fine-grained permissions like `invoice:approve`, `invoice:void`, `user:invite`. The lead wants "one service that everyone imports" but is unsure whether roles or permissions should be the primary unit the UI checks against.

Question: How would you architect the core authorization primitive for this app, and why should the UI check permissions rather than roles directly?

Answer: Roles are a convenient grouping for administrators to assign, but they are an implementation detail that changes often (a new "billing-clerk" role might appear next quarter). If every component checks `hasRole('admin')`, then adding a new role that should also see the approve button means touching every template. Instead, roles should be resolved server-side (or at login) into a flat set of *permissions*, and the UI should only ever ask "can this user do X?" — never "is this user role Y?".

Model it with a signal-based service so permission checks are reactive and integrate cleanly with `computed()` and control flow (`@if`):

```
import { Injectable, computed, signal } from '@angular/core';

export interface AuthContext {
  userId: string;
  tenantId: string;
  roles: string[];
  permissions: string[]; // flattened, e.g. ['invoice:approve', 'invoice:void']
}

@Injectable({ providedIn: 'root' })
export class AuthorizationService {
  private readonly _context = signal<AuthContext | null>(null);

  readonly context = this._context.asReadonly();
  readonly permissionSet = computed(
    () => new Set(this._context()?.permissions ?? [])
  );

  setContext(ctx: AuthContext): void {
    this._context.set(ctx);
  }

  clear(): void {
    this._context.set(null);
  }

  can(permission: string): boolean {
    return this.permissionSet().has(permission);
```

```
  }

  canAny(permissions: string[]): boolean {
    const set = this.permissionSet();
    return permissions.some(p => set.has(p));
  }

  canAll(permissions: string[]): boolean {
    const set = this.permissionSet();
    return permissions.every(p => set.has(p));
  }
}
```

Roles still matter for admin screens ("assign role X to this user") but the *product* code — guards, directives, buttons — should be permission-based. This decouples "who can approve invoices" (a business/role question, changeable in an admin panel) from "what does the approve button check" (a stable code contract). It also makes testing trivial: tests set a permission array, not a fake role hierarchy.

Critically, none of this replaces server-side checks. The permission list arrives from the server (typically embedded in the JWT or fetched from a `/me` endpoint) and every mutating API call must re-validate authorization independently — the Angular layer only controls what's *rendered*, never what's *allowed*.

Interviewer intent: Tests whether the candidate defaults to modeling authorization around roles (a common anti-pattern) or understands the roles→permissions flattening pattern used in real enterprise systems.

Scenario 2: A user with multiple roles across multiple tenants

Situation: Your SaaS product lets one user account belong to several organizations ("tenants"). Inside Tenant A the user is an `admin`; inside Tenant B they are a `viewer`. The app has a tenant switcher in the header. QA reports that after switching tenants, the sidebar still shows admin-only links from the previous tenant for a split second, and occasionally permanently if the switch fails partway.

Question: How do you model per-tenant roles/permissions so that switching tenants can't leak the previous tenant's authorization state, even transiently?

Answer: The bug is a classic "stale composite state" problem — permissions and tenant ID live in the same object but are updated in two steps (clear old, fetch new), leaving a window where old permissions render against the new tenant, or vice versa. The fix is to treat tenant + permissions as a single atomic unit that is only ever swapped, never partially mutated, and to gate the UI on a "context is loading" state during the swap.

```
import { Injectable, computed, signal } from '@angular/core';
import { firstValueFrom } from 'rxjs';

export interface TenantContext {
  tenantId: string;
  roles: string[];
  permissions: string[];
}

type AuthState =
```

```
  | { status: 'idle' }
  | { status: 'loading'; tenantId: string }
  | { status: 'ready'; context: TenantContext }
  | { status: 'error'; tenantId: string };

@Injectable({ providedIn: 'root' })
export class TenantAuthService {
  private readonly _state = signal<AuthState>({ status: 'idle' });

  readonly state = this._state.asReadonly();

  readonly isSwitching = computed(() => this._state().status === 'loading');

  readonly permissionSet = computed(() => {
    const s = this._state();
    return s.status === 'ready' ? new Set(s.context.permissions) : new Set<string>();
  });

  async switchTenant(tenantId: string, api: TenantApi): Promise<void> {
    // Atomic swap: never touch the old permission set while fetching the new one.
    this._state.set({ status: 'loading', tenantId });
    try {
      const context = await firstValueFrom(api.getContextForTenant(tenantId));
      // Guard against a race: only commit if this is still the latest request.
      if (this._state().status === 'loading' && (this._state() as any).tenantId ===
tenantId) {
        this._state.set({ status: 'ready', context });
      }
    } catch {
      this._state.set({ status: 'error', tenantId });
    }
  }

  can(permission: string): boolean {
    return this.permissionSet().has(permission);
  }
}
```

In the shell template, the sidebar is driven entirely off `permissionSet()`, and the whole nav renders a skeleton while `isSwitching()` is true — it never shows a mix of old and new. Because `_state` is a single discriminated-union signal rather than separate `tenantId` and `permissions` signals, there is no intermediate render where one has updated and the other hasn't (Angular's signal change detection can't "see" a half-updated pair, because there is no pair).

The stale-request guard (`if (... still the latest request)`) also prevents a slow response from Tenant A landing *after* the user has already switched to Tenant C, which would otherwise silently grant Tenant A's permissions in Tenant C's UI. Server-side, every API call must include the tenant ID and the backend must independently verify the user's role within *that* tenant — the client's tenant switcher is UX convenience, not a security boundary.

Interviewer intent: Probes whether the candidate can reason about race conditions and atomicity in reactive state, not just "how do I store two roles" — a very common real-world multi-tenant bug.

Scenario 3: Role changes granted by an admin don't reflect until re-login

Situation: An admin promotes a user to `manager` mid-day. The user complains they still can't see the manager features even after refreshing the page. Support's workaround is "log out and log back in," which frustrates customers. The roles are baked into a long-lived JWT access token (30-minute expiry) issued at login.

Question: Why does this happen, and what are your options to make role changes take effect promptly without compromising the stateless-JWT model?

Answer: The JWT is a *snapshot* of the user's roles at issue time — that's the entire point of a stateless token, it avoids a DB lookup on every request. Refreshing the page doesn't help because Angular just re-reads the same stale token from storage; the client never re-contacts the server for fresh claims until the token is refreshed via the refresh-token flow, and even the refreshed access token was minted from a session that hasn't re-checked the DB either, depending on implementation.

There are three realistic fixes, in order of preference:

1. **Short-lived access tokens + refresh-token rotation that re-reads roles on every refresh.** Keep access tokens very short (2–5 min). Every silent refresh call hits the auth server, which re-fetches current roles from the DB and mints a new token. Angular's `HttpInterceptorFn` already refreshes on 401/expiry — this makes role propagation bounded by the token TTL, not by "log out and back in."
2. **Server-push invalidation.** On a role change, the backend emits an event (websocket, SSE, or a lightweight polling `GET /me/version`) that tells connected clients "your permissions changed, re-fetch." The client force-refreshes its token/permission set immediately instead of waiting for TTL expiry.
3. **Permissions fetched separately from the JWT, on a short-poll or on-focus basis.** Keep the JWT for authentication only (who you are), and treat authorization (what you can do) as a separate, frequently-refreshed resource.

Here's the on-focus revalidation approach, which is cheap to add and covers the "admin changed my role in another tab" case well:

```
import { inject, DestroyRef, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class PermissionRevalidator {
  private readonly http = inject(HttpClient);
  private readonly auth = inject(AuthorizationService);

  start(): void {
    // Re-check permissions when the tab regains focus or every 60s while active.
    document.addEventListener('visibilitychange', () => {
      if (document.visibilityState === 'visible') this.revalidate();
    });
    setInterval(() => {
      if (document.visibilityState === 'visible') this.revalidate();
    }, 60_000);
  }
```

```
  private revalidate(): void {
    this.http.get<AuthContext>('/api/me/context').subscribe(ctx => {
      this.auth.setContext(ctx);
    });
  }
}
```

Whichever mechanism you pick, set expectations correctly: the *client* deciding it now has a new permission is only a UI convenience. The *server* must still independently authorize every request against the current DB state — so even in the worst case (stale client token, user clicks a now-forbidden button), the backend returns 403 and the UI should handle that gracefully (see Scenario 7) rather than trusting its own cached permission set as ground truth.

Interviewer intent: Tests understanding of JWT statelessness trade-offs and whether the candidate reaches for "force logout" as the only tool, versus designing graceful revalidation.

Scenario 4: Hide the button vs. disable it with a tooltip

Situation: Product design debates whether the "Delete Project" button should be completely hidden for users without `project:delete`, or shown-but-disabled with a tooltip explaining why. Different teams want different behavior for different actions across the app, and currently it's inconsistent — some buttons vanish, some are grayed out, with no clear rule.

Question: What criteria would you use to decide, per-action, whether to hide or disable-with-tooltip, and how would you implement both patterns cleanly with a single reusable directive?

Answer: This is a UX/discoverability decision, not a security one (both are equally "fake" from a security standpoint since the server enforces the real rule). The criteria I use:

- **Hide** when the action's mere existence is sensitive (e.g., a `viewer` shouldn't even know there's a "financial report" tab) or when the control is common/low-value and clutter matters more than discoverability (e.g., a bulk-action toolbar with 15 icons).
- **Disable + tooltip** when the user *should* know the capability exists so they can request access, or when they might legitimately gain access soon (e.g., "Approve" is disabled with "Only approvers can perform this action — ask your manager for the Approver role") — this reduces support tickets ("why can't I find X") more than hiding does.
- **Never silently hide primary/expected actions** a user reasonably assumes they have (e.g., "Save my own draft") — that reads as a bug, not a permission boundary.

Build one structural-ish attribute directive with a mode input, backed by the signal-based `AuthorizationService` from Scenario 1:

```
import { Directive, ElementRef, Renderer2, computed, effect, inject, input } from
'@angular/core';
import { AuthorizationService } from './authorization.service';

@Directive({
  selector: '[appRequiresPermission]',
  standalone: true,
})
```

```ts
export class RequiresPermissionDirective {
  private readonly auth = inject(AuthorizationService);
  private readonly el = inject(ElementRef<HTMLElement>);
  private readonly renderer = inject(Renderer2);

  readonly appRequiresPermission = input.required<string | string[]>();
  readonly mode = input<'hide' | 'disable'>('hide');
  readonly deniedTooltip = input<string>('You do not have permission to perform this action.');

  private readonly allowed = computed(() => {
    const perms = this.appRequiresPermission();
    return Array.isArray(perms) ? this.auth.canAny(perms) : this.auth.can(perms);
  });

  constructor() {
    effect(() => {
      const isAllowed = this.allowed();
      const el = this.el.nativeElement;

      if (this.mode() === 'hide') {
        this.renderer.setStyle(el, 'display', isAllowed ? '' : 'none');
        return;
      }

      // disable mode
      if (isAllowed) {
        this.renderer.removeAttribute(el, 'disabled');
        this.renderer.removeAttribute(el, 'title');
        this.renderer.removeAttribute(el, 'aria-disabled');
      } else {
        this.renderer.setAttribute(el, 'disabled', 'true');
        this.renderer.setAttribute(el, 'aria-disabled', 'true');
        this.renderer.setAttribute(el, 'title', this.deniedTooltip());
      }
    });
  }
}
```

Usage:

```ts
<button appRequiresPermission="project:delete" mode="hide">Delete Project</button>

<button
  appRequiresPermission="invoice:approve"
  mode="disable"
  deniedTooltip="Only users with the Approver role can approve invoices.">
  Approve
</button>
```

One implementation detail worth calling out in an interview: for `mode="disable"`, don't just add a CSS class that visually grays the button while leaving the click handler wired — always set the actual `disabled` attribute (or intercept `(click)` and no-op) so keyboard/automation can't trigger it, and pair it with `aria-disabled` for screen readers. And regardless of mode, the backend must reject the action independently — a disabled button is a UX hint, not an access-control mechanism; anyone can flip `disabled` off in devtools.

Interviewer intent: Checks whether the candidate treats "hide vs. disable" as a real UX design decision with criteria (not just a coin flip), and whether they can generalize both behaviors into one reusable directive instead of duplicating logic.

Scenario 5: Permission-check performance problem with hundreds of directive instances

Situation: A dense data-grid renders 500 rows, each with 4 action icons (edit/delete/approve/export), each gated by a permission directive like the one in Scenario 4. Profiling shows a noticeable jank whenever the user's permission set changes (e.g., after tenant switch) — 2,000 directive instances all recompute and touch the DOM at once, and the effect scheduling causes long tasks.

Question: How would you redesign this for performance without regressing correctness, given permissions rarely change but must still be reactive?

Answer: The core issue is redundant, uncoordinated work: 2,000 independent `effect()`s each doing their own `Set.has()` lookup and their own `Renderer2` DOM write, all triggered in the same tick, each paying scheduling overhead. Several complementary fixes, roughly in order of impact:

1. **Precompute a `computed()` per distinct permission, not per row.** There are only 4 distinct permissions in this grid (edit/delete/approve/export), not 2,000. Hoist the reactive computation to the parent/grid component as four `computed()` signals, and have each row read the *already-resolved boolean* via `@if`/property binding rather than each icon independently asking the permission service.

```
@Component({ /* ... */ })
export class DataGridComponent {
  private readonly auth = inject(AuthorizationService);

  // Computed once, shared by every row via the template — not once per row.
  readonly canEdit = computed(() => this.auth.can('row:edit'));
  readonly canDelete = computed(() => this.auth.can('row:delete'));
  readonly canApprove = computed(() => this.auth.can('row:approve'));
  readonly canExport = computed(() => this.auth.can('row:export'));
}
```

```
@for (row of rows(); track row.id) {
  <tr>
    <td>
      @if (canEdit()) { <button (click)="edit(row)">Edit</button> }
      @if (canDelete()) { <button (click)="delete(row)">Delete</button> }
    </td>
  </tr>
}
```

This eliminates the directive-per-cell entirely for the common "row-independent" case (permission doesn't vary per row) — Angular's signal-based `@if` handles insertion/removal natively and efficiently, with no custom `effect()` or `Renderer2` code needed at all.

2. **When permission genuinely depends on row data** (e.g., "can approve only your own team's rows"), keep it row-scoped but make the *directive itself* cheap: avoid `effect()` + manual DOM mutation in favor of structural control flow, which batches better and avoids re-running a full effect body (including tooltip attribute writes) on every signal tick.
3. **Batch/microtask-defer bulk permission-set swaps.** If the whole permission set changes at once (tenant switch), that's one signal write, so Angular already coalesces the resulting re-renders into a single change-detection pass (signals + zoneless or default CD both batch within a microtask) — the earlier jank was from 2,000 *separate* imperative DOM writes fighting the renderer, not from 2,000 separate recomputations. Removing the manual `Renderer2` calls (fix #1) is what actually removes the jank, because `@if` view creation/destruction is handled by Angular's own optimized reconciliation rather than 2,000 individual style mutations.
4. **Virtual scrolling** (`@angular/cdk/scrolling`) if 500 rows are rendered but only ~20 are visible — this is a broader perf fix but compounds nicely, since permission checks only run for rendered rows.

The general lesson: a directive-per-instance pattern is fine at small scale but doesn't scale to grids; hoist permission checks to the highest scope where the value is actually shared, and prefer Angular's built-in `@if`/`@for` reactivity over custom imperative DOM manipulation in perf-sensitive lists.

Interviewer intent: Tests whether the candidate can diagnose a signals/change-detection performance issue concretely (not just say "use OnPush") and knows to hoist shared computations rather than duplicating them per list item.

Scenario 6: Dynamic navigation menu built per permission set

Situation: The left nav must show a different set of menu items (and different ordering/grouping) depending on the logged-in user's permissions — some items require a single permission, some require any-of a set, some require all-of a set, and some are entire sections that should collapse away if none of their children are visible.

Question: Design a data-driven navigation model and rendering approach that avoids a giant `@if/@else if` chain in the nav template and scales as new menu items are added.

Answer: Hard-coding `@if (auth.can('a') || auth.can('b'))` per menu item doesn't scale and mixes "what the menu looks like" with "who can see what." Instead, model the menu as declarative data — a tree of nodes each carrying a permission *requirement expression* — and derive the *visible* tree as a single `computed()` that both filters leaf items and collapses empty parent sections.

```
export type PermissionRequirement =
  | { type: 'any'; permissions: string[] }
  | { type: 'all'; permissions: string[] }
  | { type: 'none' }; // always visible, e.g. "Home"

export interface NavNode {
  id: string;
```

```
  label: string;
  route?: string;
  icon?: string;
  requirement: PermissionRequirement;
  children?: NavNode[];
}

export const NAV_TREE: NavNode[] = [
  { id: 'home', label: 'Home', route: '/', requirement: { type: 'none' } },
  {
    id: 'billing',
    label: 'Billing',
    requirement: { type: 'any', permissions: ['invoice:read', 'invoice:approve'] },
    children: [
      { id: 'invoices', label: 'Invoices', route: '/billing/invoices',
        requirement: { type: 'any', permissions: ['invoice:read'] } },
      { id: 'approvals', label: 'Approvals', route: '/billing/approvals',
        requirement: { type: 'all', permissions: ['invoice:approve', 'invoice:read'] } },
    ],
  },
  {
    id: 'admin',
    label: 'Administration',
    requirement: { type: 'any', permissions: ['user:invite', 'role:manage'] },
    children: [
      { id: 'users', label: 'Users', route: '/admin/users',
        requirement: { type: 'any', permissions: ['user:invite'] } },
      { id: 'roles', label: 'Roles', route: '/admin/roles',
        requirement: { type: 'any', permissions: ['role:manage'] } },
    ],
  },
];
```

```
@Injectable({ providedIn: 'root' })
export class NavMenuService {
  private readonly auth = inject(AuthorizationService);

  private satisfies(req: PermissionRequirement): boolean {
    switch (req.type) {
      case 'none': return true;
      case 'any': return this.auth.canAny(req.permissions);
      case 'all': return this.auth.canAll(req.permissions);
    }
  }

  // Recursively filter: a parent survives only if it satisfies its own
  // requirement AND has at least one visible child (or no children at all).
  private filterNode(node: NavNode): NavNode | null {
    if (!this.satisfies(node.requirement)) return null;

    if (!node.children?.length) {
      return node;
```

```
    }

    const visibleChildren = node.children
      .map(child => this.filterNode(child))
      .filter((c): c is NavNode => c !== null);

    return visibleChildren.length > 0 ? { ...node, children: visibleChildren } : null;
  }

  readonly visibleTree = computed(() =>
    NAV_TREE.map(n => this.filterNode(n)).filter((n): n is NavNode => n !== null)
  );
}
```

The template becomes a single recursive component (`<app-nav-node [node]="n" />` calling itself for `node.children`), with zero permission-specific branching in the template itself — all the logic lives in `filterNode`, unit-testable in isolation with fabricated permission sets. Adding a new menu item is a data change, not a template change. This also naturally solves "collapse the Admin section if none of its children are visible" — a common bug in ad-hoc implementations where the parent link shows even though every child was hidden.

As always, routes behind these nav items must be independently protected by a `CanMatch` / `CanActivate` guard (Scenario 8) — the nav model only controls what's *shown*, and a user could type the URL directly.

Interviewer intent: Evaluates whether the candidate can design a scalable, declarative, testable data model for a real UI problem rather than writing more conditional spaghetti in templates.

Scenario 7: Role revoked mid-session — handling the 403 gracefully

Situation: A user is actively editing an invoice when an admin revokes their `invoice:edit` permission in another session. The user's Angular app still thinks they have permission (stale client cache) and submits the edit. The API correctly returns `403 Forbidden`. Currently the app shows a generic "Something went wrong" toast and the user loses their edits.

Question: Design the end-to-end handling for this "authorization revoked mid-session" case — both the immediate error UX and how the client resyncs its permission state.

Answer: This is the sharpest illustration of "client-side permission checks are UX, server-side is truth." Since the client's `AuthorizationService` can be stale by design (Scenario 3), a 403 on an action the UI *thought* was allowed is an expected, first-class case — not a generic error — and must be handled specifically, not swallowed into a catch-all toast.

Handle it centrally with an `HttpInterceptorFn` so every feature gets consistent behavior without repeating logic:

```
import { HttpErrorResponse, HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError, switchMap } from 'rxjs';
import { AuthorizationService } from './authorization.service';
import { NotificationService } from './notification.service';
```

```ts
import { HttpClient } from '@angular/common/http';

export const authorizationSyncInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthorizationService);
  const notify = inject(NotificationService);
  const http = inject(HttpClient);

  return next(req).pipe(
    catchError((err: unknown) => {
      if (err instanceof HttpErrorResponse && err.status === 403) {
        // 1. Resync permissions immediately — the local cache is proven stale.
        http.get<AuthContext>('/api/me/context').subscribe(ctx => auth.setContext(ctx));

        // 2. Surface a specific, actionable message instead of a generic error.
        notify.warn(
          'Your permissions changed and this action is no longer available. ' +
          'Your unsaved changes have been preserved — please review and try again.'
        );
      }
      return throwError(() => err);
    })
  );
};
```

On the component side, the save handler must preserve the user's in-progress edits rather than discarding the form, and should visually flip the UI into "read-only, permission lost" mode immediately once the resync completes (the `canEdit` computed signal used by `[disabled]` on the Save button will now correctly evaluate to false, since it derives from the same `AuthorizationService` the interceptor just updated):

```ts
save(): void {
  this.invoiceApi.update(this.invoice()).subscribe({
    next: () => this.notify.success('Saved'),
    error: (err: HttpErrorResponse) => {
      if (err.status === 403) {
        // Keep form data intact; let the user copy it out or wait for regranted access.
        this.readOnlyDueToRevokedAccess.set(true);
      }
    },
  });
}
```

This turns a confusing data-loss bug into a transparent, trustworthy experience: the user understands *why* the save failed, their work isn't lost, and the app's permission cache self-heals immediately rather than repeating the same failed request. It also closes a subtle security-adjacent UX gap — without the resync, the UI would keep showing "Edit" as enabled and let the user hit the same 403 repeatedly, which looks like a bug rather than an intentional access change.

Interviewer intent: Distinguishes candidates who treat authorization errors as generic HTTP errors from those who design a coherent recovery flow — a strong signal for production RBAC maturity.

Scenario 8: Route protection with functional CanMatch vs CanActivate

Situation: The app has `/admin/**` routes that should be completely invisible to non-admins — not just blocked with a redirect, but such that the route doesn't even "exist" for route-matching purposes (so a similarly-named public route can take over the same path segment for non-admins, and lazy chunks for admin-only feature areas aren't even requested).

Question: Why is `CanMatch` the better tool here than `CanActivate`, and how would you implement both the guard and the lazy-loaded route config?

Answer: `CanActivate` runs *after* the router has already committed to matching a route — the route config is selected, then the guard decides whether activation is allowed, and on failure you typically redirect. `CanMatch` runs *during* route resolution, before a route is chosen: if it returns false, the router treats that route config as if it didn't match at all and continues trying other configs (including a fallback `**` route). This means:

- The admin route's lazy chunk is never fetched for a non-admin (no wasted network request, and no reverse-engineering hint that an admin bundle exists).
- You can define two routes on the *same path* — an admin-capable one and a fallback — and `CanMatch` naturally picks the right one instead of you hand-rolling a redirect.

```
import { inject } from '@angular/core';
import { CanMatchFn, Router } from '@angular/router';
import { AuthorizationService } from './authorization.service';

export function permissionCanMatch(required: string | string[]): CanMatchFn {
  return () => {
    const auth = inject(AuthorizationService);
    const perms = Array.isArray(required) ? required : [required];
    return auth.canAll(perms);
  };
}
```

```
export const routes: Routes = [
  {
    path: 'admin',
    canMatch: [permissionCanMatch(['admin:access'])],
    loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),
  },
  {
    // Same path, no permission required — catches non-admins landing on /admin.
    path: 'admin',
    loadComponent: () => import('./shared/not-authorized.component')
      .then(m => m.NotAuthorizedComponent),
  },
  { path: '**', loadComponent: () => import('./shared/not-found.component').then(m =>
m.NotFoundComponent) },
];
```

Use `CanActivate` / `CanActivateChild` for cases where the route *does* exist and is matched, but you want a conditional redirect with more context (e.g., "you can see this route exists, but you're redirected to an upgrade page because your plan lacks this feature") — that's a legitimate, different UX from "this route effectively doesn't exist for you."

Both, of course, are navigation-level gates only. The `admin.routes` lazy chunk, once loaded, still needs component-level permission checks (in case permissions change after the chunk is cached) and the admin APIs it calls must independently enforce authorization server-side — client-side routing guards prevent *rendering*, not malicious direct API calls.

Interviewer intent: Confirms the candidate knows the semantic and performance difference between `CanMatch` and `CanActivate` — a frequently-confused pair, and a strong signal of current Angular router fluency.

Scenario 9: Admin impersonation feature design

Situation: Support needs an "impersonate user" feature so admins can see the app exactly as a specific customer sees it, to debug a reported issue. Leadership is nervous about security and audit implications. You're asked to design the client-side behavior.

Question: How would you design impersonation so it's safe, clearly visible to the admin, auditable, and doesn't accidentally grant the admin elevated privileges *as* the impersonated user (or vice versa)?

Answer: Impersonation is one of the highest-risk features in any RBAC system because it deliberately blurs "who is acting" — so the design goal is: the client renders *as* the impersonated user (their permission set, their data), but every server call must be traceable to the *real* admin identity, and the admin's own elevated permissions must never leak into the impersonated session.

Key design points:

1. **The impersonation token is a distinct, separate credential** — obtained via an explicit backend endpoint (`POST /admin/impersonate/{userId}`) that returns a short-lived token scoped to *only* that user's actual permissions (not unioned with the admin's). The backend must log this issuance (who impersonated whom, when, and — critically — log every subsequent request made under that token with both the acting-admin ID and the impersonated-user ID, e.g. via a claim like `actingAs: adminId`).
2. **Client-side, keep it unmistakably visible and reversible:**

```
export interface AuthContext {
  userId: string;
  roles: string[];
  permissions: string[];
  impersonation?: { active: true; actingAdminId: string; actingAdminName: string };
}

@Injectable({ providedIn: 'root' })
export class ImpersonationService {
  private readonly auth = inject(AuthorizationService);
  private readonly http = inject(HttpClient);

  readonly isImpersonating = computed(() => !!this.auth.context()?.impersonation);
```

```
  startImpersonation(targetUserId: string) {
    return this.http.post<AuthContext>('/admin/impersonate', { targetUserId }).pipe(
      tap(ctx => this.auth.setContext(ctx)),
    );
  }

  stopImpersonation() {
    return this.http.post<AuthContext>('/admin/impersonate/stop', {}).pipe(
      tap(ctx => this.auth.setContext(ctx)), // restores the real admin context
    );
  }
}
```

```
@if (impersonation.isImpersonating()) {
  <div class="impersonation-banner" role="alert">
    Viewing as {{ auth.context()?.userId }} — impersonated by
    {{ auth.context()?.impersonation?.actingAdminName }}.
    <button (click)="stopImpersonating()">Exit impersonation</button>
  </div>
}
```

The banner is persistent (not dismissible), high-contrast, and present on every route while impersonating — this is as much a safety rail for the admin (so they don't forget which account they're acting as and, say, submit a real support reply as the customer by mistake) as it is a transparency measure.

3. **Never merge permission sets.** The impersonated session's `permissions` array must be *exactly* the target user's permissions — not the admin's permissions unioned in "just in case." If the admin needs to perform an admin action while investigating, they should exit impersonation first. This prevents the impersonation feature from becoming an unintended privilege-escalation backdoor.
4. **Restrict what impersonation can do.** Typically impersonation should be read-mostly (view data, reproduce a bug) with destructive/financial actions (delete account, change payment method) disabled even if the impersonated user could normally do them — enforced server-side by checking the `actingAs` claim and blocking a deny-list of action types when it's present.
5. **Time-bound and audit everything.** Short TTL (e.g., 15 minutes, renewable with re-justification), and every action taken during impersonation is written to an audit log queryable by user, admin, and time range — this is usually a compliance/legal requirement, not just a nice-to-have.

Interviewer intent: A senior-level question testing whether the candidate thinks about impersonation as a security-sensitive feature with audit and privilege-isolation requirements, not just "swap the current user object."

Scenario 10: Feature flags vs. permissions — where's the line?

Situation: The team already has a feature-flag system (LaunchDarkly-style) for gradual rollouts. A new requirement — "only Enterprise-plan admins should see the new Analytics Dashboard" — gets implemented as a feature flag targeted at a user segment, and a developer asks whether this should instead be a permission.

Question: How do you distinguish "this belongs in the feature-flag system" from "this belongs in the RBAC/permission system," and what problems arise from conflating them?

Answer: Feature flags and permissions solve different problems and have different lifecycles, even though both end up as "should this render." The distinguishing questions:

- **Is it time-bound / experimental?** Feature flags are for things that will eventually be 100% on (or removed) — a rollout, an A/B test, a kill switch. Permissions are durable, long-lived business rules ("Enterprise admins can see Analytics" isn't going away).
- **Does it need to be queried/audited as "access control"?** Compliance and audit tooling usually needs to answer "who can see customer financial data" as a permission query. If that fact lives inside a flag-targeting rule in a third-party flagging tool, it's invisible to your access-audit process and to your own RBAC admin UI.
- **Does the rule depend on identity/role, or on rollout percentage/environment?** "10% of users, weighted by cohort" is inherently a flag concept. "Users with role X in plan Y" is inherently a permission/entitlement concept, even if it's *also* implemented as boolean gating in code.
- **Who manages it day to day?** Permissions are typically managed by customer admins or your ops/support team via a roles UI; flags are managed by engineering/product during a launch.

In this scenario, "Enterprise-plan admins see Analytics" is really an **entitlement** (plan-based permission), not a flag — model it as a permission derived from plan + role (`analytics:view` granted server-side when `plan === 'enterprise' && role === 'admin'`), so it's auditable, testable, and consistent with every other permission check in the app:

```
readonly canViewAnalytics = computed(() => this.auth.can('analytics:view'));
```

If you conflate the two — using the flag system for durable entitlements — you get real problems: the flag config becomes a second, shadow source of truth for access control that your RBAC admin panel doesn't know about; QA and audits can't answer "who has access to Analytics" from one place; and removing/renaming a flag (routine flag-system hygiene) can silently revoke access in production. Conversely, using true short-lived rollout flags as if they were permissions means they never get cleaned up and pollute the permission model with one-off, non-reusable checks.

A reasonable hybrid: allow a feature flag to *gate a permission's rollout* (e.g., `analytics:view` is only granted if both the entitlement rule *and* the `analytics-ga` flag are true) during the rollout period, then delete the flag check once fully launched — keeping the flag layer temporary and the permission layer as the durable, auditable one.

Interviewer intent: Tests architectural judgment about where different kinds of "gating" logic belong — a subtle distinction that, done wrong, causes long-term auditability and maintenance pain.

Scenario 11: Server-driven UI permissions vs. client-computed permissions

Situation: A new backend engineer proposes that instead of the client computing `canAny` / `canAll` from a flat permission list, the server should send a pre-computed UI descriptor: `{ showDeleteButton: true, showApproveButton: false, ... }` tailored to each screen. The frontend team is split on whether this is better.

Question: What are the trade-offs of server-computed "UI capability flags" versus client-computed permission checks from a generic permission list?

Answer: Both are legitimate patterns used in real systems (this is often called a BFF/"UI capabilities" pattern), and the right choice depends on how complex and screen-specific the authorization logic is.

Server-computed UI flags (`{ showDeleteButton: true }`) — pros:

- Business logic like "can delete only if invoice is unlocked AND user has invoice:delete AND invoice isn't older than 90 days" lives in one place (the server), avoiding logic duplication/drift between client and API.
- The client becomes "dumb" — just renders booleans — reducing risk of the client getting the rule wrong or out of sync with a rule change.
- Naturally handles resource-instance-level rules (row-level security) that a flat global permission list can't express well.

Server-computed UI flags — cons:

- Combinatorial explosion: every screen/button needs its own server-computed flag, and adding a button means a backend change + API contract change, slowing frontend iteration.
- Harder to reuse the *same* underlying permission across many independent UI elements (nav, buttons, tooltips, route guards) without either re-fetching a huge flags-blob per screen or duplicating flag names everywhere.
- Makes client-side testing/storybook work harder — you must mock a large bespoke flags object per scenario instead of toggling a couple of permission strings.

Client-computed from a flat permission list — pros:

- One generic contract (`permissions: string[]`) reused everywhere: nav, guards, directives, buttons — the pattern from Scenarios 1, 4, 6, 8 all compose off it.
- Cheap to test and reason about; new UI elements don't require new API fields.

Client-computed — cons:

- Instance-level rules ("delete allowed only if this specific invoice is unlocked") don't fit a static permission list well — you end up mixing in resource state (`invoice.status === 'unlocked'`) with permission checks in the component, which can drift from the server's real rule.

The pattern I'd actually recommend is a hybrid: use a flat global permission list (client-computed) for *coarse* screen/feature/nav-level gating, and use server-returned per-resource capability flags embedded *in the resource payload itself* for fine-grained, instance-level actions:

```
export interface Invoice {
  id: string;
  amount: number;
  status: 'draft' | 'unlocked' | 'locked';
  // Server computes this alongside the resource — combines permission + business state.
  capabilities: { canEdit: boolean; canDelete: boolean; canApprove: boolean };
}
```

```
@if (invoice().capabilities.canDelete) {
  <button (click)="delete(invoice())">Delete</button>
}
```

This keeps the *global* permission model simple and reusable (for nav/guards) while letting the server own row-level, stateful authorization logic exactly where the resource's data already lives — no separate flags-blob endpoint, no combinatorial explosion, and no risk of the client re-deriving business rules the server should own.

Interviewer intent: Assesses architectural maturity around where authorization *logic* (not just checks) should live, especially for row-level/instance-level rules that a naive global permission list can't express.

Scenario 12: Structural directive with signal-based state — `*hasPermission`

Situation: The team wants a structural directive `*hasPermission="'invoice:approve'"` (similar to `*ngIf`) that also supports an `else` template, integrates with signals for reactivity, and — unlike the attribute directive in Scenario 4 — actually adds/removes the element from the DOM (not just hides it), for cases where hiding via CSS still leaves inert-but-present elements that a screen reader or fast typist could interact with.

Question: Implement this structural directive using modern Angular APIs (signals, `input()`, embedded views) with `else` support.

Answer: A structural directive fundamentally manages an `<ng-template>` via a `ViewContainerRef`, deciding whether to instantiate it. Building it around signals means the show/hide logic reacts automatically to permission changes without manual subscription management:

```
import {
  Directive, EmbeddedViewRef, Injector, TemplateRef, ViewContainerRef,
  effect, inject, input,
} from '@angular/core';
import { AuthorizationService } from './authorization.service';

@Directive({
  selector: '[hasPermission]',
  standalone: true,
})
export class HasPermissionDirective {
  private readonly templateRef = inject(TemplateRef<unknown>);
  private readonly viewContainer = inject(ViewContainerRef);
  private readonly auth = inject(AuthorizationService);
  private readonly injector = inject(Injector);

  // Bindings: *hasPermission="perm; else noAccessTpl"
  readonly hasPermission = input.required<string | string[]>({ alias: 'hasPermission' });
  readonly hasPermissionElse = input<TemplateRef<unknown> | null>(null);

  private thenViewRef: EmbeddedViewRef<unknown> | null = null;
  private elseViewRef: EmbeddedViewRef<unknown> | null = null;
```

```
  constructor() {
    // Run inside an injection-context effect so it re-executes whenever
    // the permission set OR the required permission input changes.
    effect(() => {
      const required = this.hasPermission();
      const allowed = Array.isArray(required)
        ? this.auth.canAny(required)
        : this.auth.can(required);

      this.viewContainer.clear();
      this.thenViewRef = null;
      this.elseViewRef = null;

      if (allowed) {
        this.thenViewRef = this.viewContainer.createEmbeddedView(this.templateRef);
      } else {
        const elseTpl = this.hasPermissionElse();
        if (elseTpl) {
          this.elseViewRef = this.viewContainer.createEmbeddedView(elseTpl);
        }
      }
    }, { injector: this.injector });
  }
}
```

Usage mirrors `*ngIf`/`else`:

```
<div *hasPermission="'invoice:approve'; else noAccess">
  <button (click)="approve(invoice())">Approve</button>
</div>
<ng-template #noAccess>
  <span class="muted">You don't have permission to approve invoices.</span>
</ng-template>
```

Compared to the CSS-hiding attribute directive from Scenario 4, this fully removes the element (and destroys its component/child instances) from the DOM and change-detection tree when not permitted — appropriate when the element is expensive (a whole panel, not just a button), when it must be unreachable to assistive tech and automated form-fillers, or when its presence (even hidden) would leak information via DOM inspection (e.g., an element attribute revealing a customer ID the viewer isn't supposed to know about). The trade-off is a slightly higher re-render cost on every permission change (destroy+recreate the view) versus a style toggle — negligible for a handful of gated sections, worth avoiding at the 2,000-instance grid scale from Scenario 5, where the lighter `@if` control-flow approach is preferable to avoid custom effect overhead per row.

Interviewer intent: Confirms hands-on fluency with writing a modern structural directive (not just consuming `*ngIf`) and whether the candidate understands when full DOM removal is warranted versus simple hide/disable.

Scenario 13: Composing multiple functional guards on one route

Situation: A route `/projects/:id/settings` needs to satisfy three independent conditions to activate: user is authenticated, user has `project:settings:view` for *that specific* project (not just globally), and the project itself is not archived (archived projects redirect to a read-only summary). The team wants these composed cleanly rather than one giant guard function.

Question: How do you compose multiple, independently-testable functional guards for one route, including a guard that needs the route's own `:id` param to check a resource-specific permission?

Answer: Angular's functional guards (`CanActivateFn`, `CanMatchFn`) are just functions, so they compose naturally — write small single-purpose guards and combine them in the route's `canActivate` array (all must pass) rather than writing one monolithic guard with three `if`s. `CanActivateFn` receives the route snapshot, which is exactly what's needed to read `:id` for a resource-scoped check:

```
import { inject } from '@angular/core';
import { ActivatedRouteSnapshot, CanActivateFn, Router, RouterStateSnapshot } from
'@angular/router';
import { map, of, catchError } from 'rxjs';
import { AuthService } from './auth.service';
import { ProjectPermissionApi } from './project-permission.api';

export const isAuthenticatedGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.isAuthenticated() ? true : router.parseUrl('/login');
};

export const projectSettingsPermissionGuard: CanActivateFn = (
  route: ActivatedRouteSnapshot,
  state: RouterStateSnapshot,
) => {
  const api = inject(ProjectPermissionApi);
  const router = inject(Router);
  const projectId = route.paramMap.get('id')!;

  // Resource-scoped check: global 'project:settings:view' isn't enough —
  // must confirm the user has it *for this specific project* (e.g., they're
  // a member of this project's team).
  return api.canViewSettings(projectId).pipe(
    map(allowed => allowed ? true : router.parseUrl('/not-authorized')),
    catchError(() => of(router.parseUrl('/not-authorized'))),
  );
};

export const projectNotArchivedGuard: CanActivateFn = (route) => {
  const api = inject(ProjectPermissionApi);
  const router = inject(Router);
  const projectId = route.paramMap.get('id')!;

  return api.getProjectStatus(projectId).pipe(
```

```
    map(status => status === 'archived'
      ? router.parseUrl(`/projects/${projectId}/summary`)
      : true),
  );
};
```

```
export const routes: Routes = [
  {
    path: 'projects/:id/settings',
    canActivate: [isAuthenticatedGuard, projectSettingsPermissionGuard,
projectNotArchivedGuard],
    loadComponent: () => import('./project-settings.component').then(m =>
m.ProjectSettingsComponent),
  },
];
```

Guards in the array run in order and short-circuit on the first one that returns a `UrlTree` (redirect) or `false`; Angular runs them and navigates to whichever redirect fires first if any do. Each guard is independently unit-testable (mock just `AuthService` for the first, just `ProjectPermissionApi` for the other two) instead of needing to stub every dependency to test one `if` branch of a monolith.

Note the resource-scoped guard calls a *server* endpoint (`api.canViewSettings(projectId)`) rather than checking a client-cached flat permission list — global permissions ("has the settings-view permission type at all") can be checked client-side instantly, but "for *this* project specifically" usually requires a server round-trip unless the client already holds a full per-resource ACL (rare, and itself would need to be kept in sync). This is a good moment in an interview to flag that resource-scoped authorization checks are inherently harder to cache client-side and often *must* hit the server — a real limitation of purely client-side RBAC.

Interviewer intent: Tests fluency with functional guard composition (a newer Angular pattern replacing class-based `CanActivate`) and awareness that resource-scoped permission checks often can't be fully resolved from a cached client permission list.

Scenario 14: Testing RBAC-gated components without a real backend

Situation: The QA/testing lead wants confidence that permission-gated UI (buttons, nav items, routes) is correct for every role combination, without spinning up a backend or logging in as different test users for every spec. There are ~15 permissions and dozens of components/directives depending on them.

Question: Design a testing strategy for RBAC-gated UI — both unit tests for individual gated components and a broader strategy for catching regressions across the whole permission matrix.

Answer: Because the whole RBAC UI layer (Scenarios 1, 4, 6, 8, 12) is deliberately built on top of one small, mockable seam — `AuthorizationService.permissionSet` — testing it doesn't require a real backend or real login at all; it requires being able to *set an arbitrary permission list* and assert on rendered output.

1. Unit/component tests — inject a fake `AuthorizationService`:

```
import { TestBed } from '@angular/core/testing';
import { AuthorizationService } from './authorization.service';
```

```
describe('InvoiceActionsComponent', () => {
  function setup(permissions: string[]) {
    const auth = {
      can: (p: string) => permissions.includes(p),
      canAny: (ps: string[]) => ps.some(p => permissions.includes(p)),
      canAll: (ps: string[]) => ps.every(p => permissions.includes(p)),
    };
    TestBed.configureTestingModule({
      imports: [InvoiceActionsComponent],
      providers: [{ provide: AuthorizationService, useValue: auth }],
    });
    const fixture = TestBed.createComponent(InvoiceActionsComponent);
    fixture.detectChanges();
    return fixture;
  }

  it('shows Approve button when invoice:approve is granted', () => {
    const fixture = setup(['invoice:approve']);
    const btn = fixture.nativeElement.querySelector('[data-testid="approve-btn"]');
    expect(btn).toBeTruthy();
  });

  it('hides Approve button when permission is missing', () => {
    const fixture = setup(['invoice:read']);
    const btn = fixture.nativeElement.querySelector('[data-testid="approve-btn"]');
    expect(btn).toBeFalsy();
  });
});
```

Since `AuthorizationService` uses signals internally but exposes plain methods (`can` / `canAny` / `canAll`), a fake implementing the same interface is trivial and doesn't need `TestBed` to fight real signal wiring.

2. Directive/guard unit tests follow the same pattern — for guards, call the `CanActivateFn` directly inside `TestBed.runInInjectionContext()` with the fake service provided, asserting the return value (`true` vs. a `UrlTree`) without any router navigation actually happening.

3. Matrix/property-style coverage — rather than hand-writing "admin sees X, viewer doesn't" per component, define the permission matrix once as data and drive parameterized tests off it:

```
const PERMISSION_MATRIX: Record<string, string[]> = {
  admin: ['invoice:read', 'invoice:approve', 'invoice:delete', 'user:invite'],
  manager: ['invoice:read', 'invoice:approve'],
  viewer: ['invoice:read'],
};

describe.each(Object.entries(PERMISSION_MATRIX))('as %s', (role, permissions) => {
  it('renders the nav correctly', () => {
    const tree = new NavMenuServiceFake(permissions).visibleTree();
    expect(tree).toMatchSnapshot(); // one snapshot file per role, reviewed on PR
  });
});
```

Snapshot-per-role for the *derived nav tree* (Scenario 6) is a high-leverage regression catch: any PR that accidentally changes what a `viewer` can see shows up as a snapshot diff in review, without needing E2E logins.

4. A thin E2E smoke layer on top (Playwright/Cypress) that logs in as 2–3 representative seeded test users (not all 15 permissions individually) to catch integration issues the unit layer can't — real token expiry, real guard redirects, real server 403 handling from Scenario 7. This layer is deliberately small because it's slow and flaky compared to the fake-service unit tests, which should carry the bulk of the permission-matrix coverage.

The overarching principle: because permission logic was designed (Scenario 1) as pure functions over a settable signal/set, testing it never needs network or a real IdP — that design choice pays for itself entirely at the testing stage.

Interviewer intent: Tests whether the candidate designs for testability from the start and can articulate a layered test strategy (unit/fake-service, guard tests, matrix snapshots, thin E2E) rather than "just write e2e tests for every role."

Scenario 15: Optimistic UI unlock before the permission check resolves

Situation: On initial app load, there's a brief window (a few hundred ms) while `/api/me/context` is in flight before `AuthorizationService` has any data. During that window, permission-gated buttons currently default to "hidden" (since `permissionSet()` is an empty `Set` until loaded), causing a visible flash where the whole toolbar is empty, then suddenly populates once permissions arrive.

Question: How do you handle the "unknown" state (context not yet loaded) distinctly from "no permission," and what's the right default UX for each type of gated element during that window?

Answer: The bug is conflating two genuinely different states — "we know you don't have this permission" and "we don't know yet" — into one falsy value (empty `Set`). They deserve different UX: a skeleton/loading placeholder for the latter, not a silent "as if denied" render, because "denied" and "loading" look identical to the user but mean very different things, and briefly showing "denied" for something the user *does* have permission for (before the real data arrives) reads as flicker/jank, or worse, momentarily confuses the user into thinking they lack access.

Model loading as an explicit state rather than an empty collection:

```
type AuthState =
  | { status: 'loading' }
  | { status: 'ready'; permissions: Set<string> }
  | { status: 'error' };

@Injectable({ providedIn: 'root' })
export class AuthorizationService {
  private readonly _state = signal<AuthState>({ status: 'loading' });
  readonly state = this._state.asReadonly();

  readonly isLoading = computed(() => this._state().status === 'loading');

  can(permission: string): boolean {
    const s = this._state();
```

```
    return s.status === 'ready' && s.permissions.has(permission);
  }
}
```

Then the gating directive/structural directive distinguishes three renders — loading, granted, denied — instead of two:

```
@switch (true) {
  @case (auth.isLoading()) {
    <div class="skeleton skeleton-button"></div>
  }
  @case (auth.can('invoice:approve')) {
    <button (click)="approve()">Approve</button>
  }
  @default {
    <!-- render nothing, or a disabled/tooltip variant per Scenario 4 -->
  }
}
```

The right default per element type:

- **Whole toolbars/nav sections:** render a skeleton placeholder matching the eventual layout's shape, so there's no visible reflow/jank once real data arrives (avoids CLS-like jumps).
- **Route guards:** block navigation resolution until `isLoading()` is false (i.e., the guard itself awaits the context fetch, using `toObservable` / `firstValueFrom` over the signal, or simply the route's resolver waits on the same `/me/context` call) — never let a route render prematurely as "denied" just because context hasn't loaded, since that could bounce a legitimately-authorized user to a "not authorized" page for a split second, which is a worse UX than a brief spinner.
- **Individual inline buttons in already-rendered content:** a lightweight opacity/skeleton pulse is usually enough since the surrounding content is already meaningful and a full-page block would be excessive.

The key design lesson: authorization state, like any async data, has a loading/ready/error lifecycle — treating "not yet known" as equivalent to "denied" is a subtle but common bug that produces confusing flicker, and modeling it as a proper tri-state (or discriminated union, as here) fixes it cleanly and makes the "unknown" case impossible to accidentally skip since TypeScript forces exhaustive handling of the union.

Interviewer intent: Checks whether the candidate recognizes "unknown" as a distinct state from "denied" in async authorization data — a subtlety many implementations get wrong by defaulting to empty/falsy collections.

Scenario 16: Row-level security in a shared list — "see all" vs "see mine"

Situation: A support-ticket list screen must show different scopes of data depending on role: a `viewer` sees only tickets they created, a `manager` sees all tickets for their team, and an `admin` sees every ticket across all teams. The naive first implementation fetched all tickets and filtered them client-side by role, which QA correctly flagged as a security bug (network tab shows tickets the viewer shouldn't even receive).

Question: What was wrong with the original design, and how should row-level, scope-based authorization be structured between client and server?

Answer: The original bug is a textbook "authorization theater" — the client received the *unfiltered* dataset over the wire and merely chose not to *render* some of it. Anyone opening devtools' Network tab (or replaying the request with curl) sees every ticket regardless of role, which is a real data leak, not just a UX inconsistency. Row-level/scope filtering must happen server-side, at the query level, never as a client-side `.filter()` on an already-fetched full dataset.

The fix: the server derives the query scope from the authenticated user's role/team context and only ever returns rows the caller is entitled to see:

```
// Client just asks for "tickets" — it does NOT know or decide the scope.
@Injectable({ providedIn: 'root' })
export class TicketApi {
  private readonly http = inject(HttpClient);

  // The server infers scope (own/team/all) from the auth token —
  // the client sends no "role" parameter that could be tampered with.
  list(): Observable<Ticket[]> {
    return this.http.get<Ticket[]>('/api/tickets');
  }
}
```

Server-side pseudocode logic (illustrating the principle, not Angular):

```
GET /api/tickets
  if user.role == 'admin': return all tickets
  elif user.role == 'manager': return tickets where team_id = user.team_id
  else: return tickets where created_by = user.id
```

The client's job shrinks to two things: (1) rendering whatever the server returns, without needing its own scope logic, and (2) optionally *labeling* the UI to reflect the active scope, purely for clarity — e.g., a manager sees a header "Showing: My Team's Tickets" — which can be a hint but must never be the mechanism restricting what data is fetched:

```
readonly scopeLabel = computed(() => {
  if (this.auth.can('ticket:view:all')) return 'All Teams';
  if (this.auth.can('ticket:view:team')) return 'My Team';
  return 'My Tickets';
});
```

If there's a genuine product need for a manager to *toggle* between "my tickets" and "my team's tickets" as a filter (not a security boundary, since they're entitled to both), that's fine as a client-driven query parameter — but the server must still independently validate that the requested scope is ⊆ what the caller is actually entitled to, every time, rather than trusting the client's requested filter value.

The broader principle for the interview: any time authorization determines *which rows* of data are visible (not just which buttons), that filtering is a data-access-layer concern and belongs in the query, not in a client-side array filter — client-side filtering of a permission-sensitive dataset is one of the most common real-world RBAC vulnerabilities, and one of the easiest to miss in code review because it "looks correct" in the UI.

Interviewer intent: A security-focused question distinguishing "the UI looks right" from "the data is actually protected" — tests whether the candidate reflexively checks where filtering happens, not just whether it happens.

Scenario 17: Combining permission checks with route data for breadcrumb/title generation

Situation: The app auto-generates page titles and breadcrumbs from route `data`. For an admin-only report page, the breadcrumb currently always says "Financial Reports," even for a user who got there transiently before a guard redirect fires, briefly flashing the restricted page's title in the browser tab and breadcrumb before the redirect completes.

Question: How do you ensure route metadata (titles, breadcrumbs, analytics page-view events) tied to a permission-gated route never leaks information before the guard has resolved?

Answer: This is a sequencing bug: Angular's `Title` strategy (or a custom breadcrumb resolver reading `route.data`) can run and update the document title/breadcrumb trail as part of route *activation processing*, and if that logic doesn't explicitly wait on the guard's own resolution, a title bound directly to static `data: { title: 'Financial Reports' }` can be set by a title strategy that runs independently of guard timing, or — more commonly — a quick flash occurs when the guard is *sync* but the redirect navigation itself takes a tick to actually swap the rendered component.

The safe pattern: never put sensitive titles/breadcrumb labels in *static* route `data` for permission-gated routes at all. Instead, resolve them through a title strategy or resolver that itself performs (or waits on) the same permission check the guard uses, so there is no code path where the sensitive label is set without the permission having already been confirmed:

```
import { Injectable, inject } from '@angular/core';
import { Title } from '@angular/platform-browser';
import { RouterStateSnapshot, TitleStrategy } from '@angular/router';
import { AuthorizationService } from './authorization.service';

@Injectable({ providedIn: 'root' })
export class PermissionAwareTitleStrategy extends TitleStrategy {
  private readonly title = inject(Title);
  private readonly auth = inject(AuthorizationService);

  override updateTitle(snapshot: RouterStateSnapshot): void {
    const deepest = this.buildTitle(snapshot);
    const requiredPermission = this.deepestRouteData(snapshot)?.['requiredPermission'] as
string | undefined;

    // Only reveal the real title if the permission is actually held;
    // otherwise fall back to a neutral title, never the sensitive one.
    const canSeeTitle = !requiredPermission || this.auth.can(requiredPermission);
    this.title.setTitle(canSeeTitle && deepest ? deepest : 'App');
  }

  private deepestRouteData(snapshot: RouterStateSnapshot) {
    let route = snapshot.root;
```

```
    while (route.firstChild) route = route.firstChild;
    return route.data;
  }
}
```

```
providers: [{ provide: TitleStrategy, useClass: PermissionAwareTitleStrategy }]
```

The same principle applies to a breadcrumb service and to analytics page-view tracking: any of them reading `route.data.title` must apply the *same* permission predicate the guard uses (ideally sourced from the same route `data.requiredPermission` key, so guard and title/breadcrumb/analytics never drift out of sync) before exposing the label, rather than trusting that "if we got this far in rendering, the guard must have passed" — which is exactly the assumption that breaks during the brief window a redirecting navigation is still resolving.

A secondary, simpler mitigation worth mentioning: since `CanMatch` (Scenario 8) prevents the route from being considered a match at all when denied, using `CanMatch` instead of `CanActivate` for permission-gated routes removes an entire class of "activation started, then aborted" timing windows, because the denied route configuration never begins activating in the first place — the fallback route's neutral title is used from the start.

Interviewer intent: A detail-oriented question testing whether the candidate thinks about information leakage in ancillary systems (titles, breadcrumbs, analytics) that are easy to forget sit downstream of the same permission boundary as the main content.

Scenario 18: Migrating from a legacy NgModule role-guard to a functional, permission-based one

Situation: A large legacy app has a class-based `RoleGuard implements CanActivate` referencing hardcoded role strings (`'ROLE_ADMIN'`) sprinkled across dozens of `NgModule`-declared route configs. Leadership wants an incremental migration to standalone components, signals, and permission-based `CanMatch` guards without a risky big-bang rewrite.

Question: Design an incremental migration path from the legacy role-based `CanActivate` guard to the modern permission-based functional `CanMatch` approach.

Answer: A big-bang rewrite of dozens of route configs and their underlying role checks is exactly the kind of change that causes production authorization regressions (accidentally under- or over-granting access), so the migration should be staged to keep old and new systems correct and cross-checked simultaneously, then removed once confidence is high.

Step 1 — introduce the permission layer alongside the existing role system, without removing it. Compute permissions as a derived view of existing roles, so nothing about the legacy role assignment changes yet:

```
// Bridges legacy roles to the new permission model without touching the backend yet.
const ROLE_TO_PERMISSIONS: Record<string, string[]> = {
  ROLE_ADMIN: ['invoice:read', 'invoice:approve', 'invoice:delete', 'user:invite'],
  ROLE_MANAGER: ['invoice:read', 'invoice:approve'],
  ROLE_VIEWER: ['invoice:read'],
};

function deriveePermissions(roles: string[]): string[] {
  return [...new Set(roles.flatMap(r => ROLE_TO_PERMISSIONS[r] ?? []))];
}
```

Step 2 — write the functional guard as a thin wrapper, and run it side-by-side (not replacing) the legacy guard on a handful of low-risk routes first, logging any disagreement between old and new decisions to catch mapping mistakes before they affect real users:

```
export function permissionCanMatch(required: string): CanMatchFn {
  return (route, segments) => {
    const auth = inject(AuthorizationService);
    const legacyGuard = inject(RoleGuard); // old guard, kept temporarily
    const newDecision = auth.can(required);

    if (typeof ngDevMode !== 'undefined' && ngDevMode) {
      const legacyDecision = legacyGuard.checkRoleSync(route);
      if (legacyDecision !== newDecision) {
        console.warn(`[migration] permission/role mismatch on ${segments.join('/')}`,
          { legacyDecision, newDecision });
      }
    }
    return newDecision;
  };
}
```

Step 3 — migrate route-by-route, converting each `NgModule`-declared feature into a standalone lazy-loaded route with `canMatch: [permissionCanMatch(...)]`, replacing `canActivate: [RoleGuard]` one module at a time — each conversion is a small, reviewable, revertible PR rather than one enormous change. Feature areas with the least traffic/risk go first to build confidence.

Step 4 — once every route has moved and the dev-mode mismatch warnings have been silent for a full release cycle (indicating the new permission derivation matches legacy behavior in production traffic), delete `RoleGuard`, the mismatch-logging wrapper, and eventually replace the hardcoded `ROLE_TO_PERMISSIONS` bridge with a real server-side permission list once the backend team ships flattened permissions in the auth payload (closing the loop back to the Scenario 1 architecture).

Throughout, keep both systems reading from the *same* underlying `AuthContext`/role data — never let the legacy and new guard diverge in *data source*, only in *how they interpret it* — so any mismatch surfaced in Step 2 is a genuine logic bug in the new mapping, not a data-sync artifact.

Interviewer intent: Tests migration/refactoring judgment on a legacy codebase — whether the candidate proposes a safe, incremental, verifiable path versus a risky rewrite, which is a very realistic senior-engineer scenario.

Scenario 19: A permission directive interacting badly with `OnPush` and content projection

Situation: A permission-gated action button is projected via `<ng-content>` into a reusable `<app-card>` component that uses `ChangeDetectionStrategy.OnPush`. After a tenant switch changes the permission set, the projected button doesn't hide even though `AuthorizationService`'s signal has updated — but it hides correctly if the user does something else that happens to trigger the card's own change detection (like resizing the window, which fires an unrelated event handler).

Question: Diagnose why an `OnPush` component with projected, permission-gated content might not react to a permission signal change, and what's the correct fix.

Answer: If the permission-gating logic were still built with the old imperative pattern (manually reading a service value inside `ngOnChanges`/a getter, rather than a genuine signal read inside a reactive context), then under `OnPush` the projected content only re-renders when the *card component itself* receives a new `@Input()` reference or when an event originating inside the card triggers CD — a signal changing elsewhere doesn't automatically schedule change detection for an `OnPush` component unless that component is actually *reading* the signal in a template expression or in an `effect()`/`computed()` that Angular is tracking.

The window-resize "fix" is a red herring/symptom: it only appears to work because resize triggers a global change-detection pass in zone-based apps, which re-evaluates the getter and *happens* to pick up the new value at that point — masking the real bug (the card isn't actually reactively bound to the signal) rather than fixing it.

The correct fix is to ensure the permission check is a genuine template-level signal read (or an `input()`/`computed()` the OnPush component's own template directly evaluates), because Angular's signal-aware change detection specifically schedules a check for a component when a signal *read directly in that component's template* changes — this is the mechanism that makes signals compatible with `OnPush` without manual `markForCheck()` calls:

```
// Bad: a getter that reads a service value imperatively, not tracked as a signal read.
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, /* ... */ })
export class CardComponent {
  private readonly auth = inject(AuthorizationService);
  get canShowAction(): boolean { // NOT tracked — OnPush won't know to re-check this
    return this.auth.can('card:action');
  }
}
```

```
// Good: expose a computed() signal, and read it directly in the template.
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, /* ... */ })
export class CardComponent {
  private readonly auth = inject(AuthorizationService);
  readonly canShowAction = computed(() => this.auth.can('card:action'));
}
```

```
<div class="card">
  <ng-content></ng-content>
  @if (canShowAction()) {
    <ng-content select="[cardAction]"></ng-content>
  }
</div>
```

Because `canShowAction()` is invoked directly in the template, Angular's reactivity graph knows this `OnPush` component depends on it, and will schedule a check the moment `AuthorizationService`'s underlying signal changes — no `markForCheck()`, no reliance on an unrelated resize event coincidentally forcing a global CD pass.

The broader lesson for an interview: `OnPush` + signals works seamlessly *only* when the signal is actually read in a tracked reactive context (template expression, `computed()`, `effect()`) — any indirection through a plain getter, a manual subscription callback that doesn't itself call `markForCheck()`, or a value copied into a plain field at construction time, silently breaks the automatic OnPush integration and reintroduces exactly the kind of "works by accident" bug described here.

Interviewer intent: A deep-dive question on the actual mechanics of signals + `OnPush` change detection — separates candidates who use signals superficially from those who understand why/when they integrate automatically with `OnPush`.

Scenario 20: Designing the permission model for a workflow with approval chains

Situation: A document approval workflow requires sequential sign-off: a document must be approved by a `reviewer` before a `manager` can approve it, and by a `manager` before a `director` can give final approval. The UI must show the correct action button (Review / Approve / Final Approve / nothing) based on both the user's role *and* the document's current workflow stage — a plain `manager` should not see "Approve" on a document still awaiting reviewer sign-off.

Question: How do you model permission checks that depend on both static role/permission grants and dynamic workflow state, without hardcoding stage logic all over the templates?

Answer: This is a case where a flat permission check (`auth.can('document:approve')`) is necessary but not sufficient — the *action available right now* is a function of (a) what the user's role is generally entitled to do, and (b) what stage the specific document is in. Conflating these into one ad hoc `@if` per template leads to duplicated, hard-to-audit stage logic scattered across every component that renders a document action button.

The clean approach: define the workflow as an explicit state machine (stages and their required role) and derive the *single next allowed action* for a given document + user as one pure function, exposed as a `computed()` off both the document signal and the user's roles:

```
export type WorkflowStage = 'draft' | 'awaiting_review' | 'awaiting_manager' |
'awaiting_director' | 'approved';

interface StageRule {
  requiredRole: string;
```

```
  actionLabel: string;
  nextStage: WorkflowStage;
}

const WORKFLOW: Partial<Record<WorkflowStage, StageRule>> = {
  awaiting_review:   { requiredRole: 'reviewer', actionLabel: 'Review & Sign Off',
nextStage: 'awaiting_manager' },
  awaiting_manager:  { requiredRole: 'manager',  actionLabel: 'Approve',
nextStage: 'awaiting_director' },
  awaiting_director: { requiredRole: 'director', actionLabel: 'Final Approve',
nextStage: 'approved' },
};

@Injectable({ providedIn: 'root' })
export class DocumentWorkflowService {
  private readonly auth = inject(AuthorizationService);

  // Pure derivation: given the document's current stage and the user's roles,
  // what single action (if any) can this user take right now?
  actionFor(document: Signal<{ stage: WorkflowStage }>) {
    return computed(() => {
      const rule = WORKFLOW[document().stage];
      if (!rule) return null; // 'draft' or 'approved' have no pending action
      return this.auth.hasRole(rule.requiredRole) ? rule : null;
    });
  }
}
```

```
@Component({ /* ... */ })
export class DocumentDetailComponent {
  private readonly workflow = inject(DocumentWorkflowService);
  readonly document = input.required<{ stage: WorkflowStage; id: string }>();

  // One computed per document instance; template only ever renders this.
  readonly pendingAction = computed(() => this.workflow.actionFor(this.document as any)());
}
```

```
@if (pendingAction(); as action) {
  <button (click)="advance(action.nextStage)">{{ action.actionLabel }}</button>
} @else {
  <span class="muted">No action required from you at this stage.</span>
}
```

This gives one authoritative place (`WORKFLOW` + `actionFor`) that answers "what can this user do to this document right now," rather than templates independently reasoning about `stage === 'awaiting_manager' && auth.hasRole('manager')`. Adding a new stage (e.g., a `compliance` sign-off inserted between manager and director) is a one-line change to the `WORKFLOW` map, not a hunt through every template that renders a document action.

Two RBAC-specific lessons this scenario tests: first, role checks here are legitimately used (rather than flat permissions) because the workflow is inherently role-sequenced by *job function*, which is a reasonable exception to the "always use permissions, not roles" guidance from Scenario 1 — when the business rule genuinely is "the *reviewer role*, specifically, signs off at this stage," modeling it as a role check is more honest than inventing a synthetic permission per stage. Second, and non-negotiably: the server must independently validate both the user's role *and* the document's current stage before accepting an approval action — a client that's momentarily out of sync (another approver just advanced the stage half a second ago) must not be able to submit a stage-inappropriate approval; the server rejects any approval that doesn't match its own authoritative view of the document's stage, and the client should treat that rejection the same way as the mid-session revocation case in Scenario 7.

Interviewer intent: Tests whether the candidate can design authorization logic that combines static grants with dynamic domain state cleanly, and recognizes when role-based (rather than permission-based) checks are actually the more appropriate modeling choice.

Quick Revision Cheat Sheet

- **Permissions, not roles, drive the UI.** Flatten roles into permissions at login/refresh; check `auth.can('resource:action')` everywhere in the UI, and reserve role checks for cases where the business rule is genuinely role-sequenced (e.g., approval chains in Scenario 20).
- **Treat tenant/permission context as one atomic unit**, never as separate signals updated in two steps — this closes the stale-permission race condition seen during tenant switching.
- **JWT-embedded roles are a snapshot, not a live value.** Role changes need short token TTLs, server-push invalidation, or on-focus/interval revalidation — "log out and back in" is a workaround, not a design.
- **Hide vs. disable-with-tooltip is a UX decision with criteria**: hide sensitive/cluttering actions; disable+explain actions the user should know exist so they can request access.
- **Hoist shared permission computations** (`computed()` at the parent/list level) instead of one directive instance per row — this is what actually fixes jank in large lists, not micro-optimizing the check itself.
- **Model navigation as declarative data with a requirement expression per node**, and derive the visible tree with one recursive `computed()` that also collapses empty parent sections.
- **A 403 on an action the client thought was allowed is expected, not exceptional** — resync permissions immediately, preserve user input, and show a specific message, never a generic error toast.
- **Use `CanMatch` (not `CanActivate`) when the route should not exist at all** for unauthorized users — it skips lazy-chunk loading and lets a same-path fallback route take over cleanly.
- **Impersonation must never union permission sets.** The impersonated session gets exactly the target user's permissions, every request is tagged with the real admin's identity for audit, and destructive actions are typically blocked outright.
- **Feature flags and permissions solve different problems** — flags are temporary/rollout-oriented, permissions are durable/auditable entitlements; conflating them creates a shadow, unauditable access-control system.
- **Global permission lists handle coarse gating; instance-level rules belong on the resource itself** (server-computed `capabilities` on the returned object), not in a combinatorial explosion of screen-specific server flags or client-side re-derivation of business rules.

- **Structural directives that fully remove elements** (vs. CSS-hide) are for expensive, sensitive, or assistive-tech-reachable content — not for high-frequency list items, where lighter `@if` control flow wins.
- **Compose small, single-purpose functional guards** rather than one monolithic guard; resource-scoped checks (permission "for this specific record") often can't be resolved from a cached client permission list and need a server round trip.
- **RBAC UI is trivially testable when built on one mockable seam** (a fake `AuthorizationService`) — drive coverage from a permission matrix and reserve real E2E logins for a thin integration-smoke layer.
- **Model "unknown" (loading) as a distinct state from "denied"** — an empty/falsy permission collection during initial load is not the same thing as a confirmed denial, and conflating them causes flicker and premature "not authorized" redirects.
- **Row/scope-level filtering ("see mine" vs "see all") must happen in the server query, never as a client-side filter over an already-fetched full dataset** — that's a real data leak, not just a UX bug.
- **Sensitive route metadata (titles, breadcrumbs, analytics) must apply the same permission predicate as the route guard**, or it can leak information during a redirect's timing window.
- **Migrate legacy role guards incrementally**: bridge roles to permissions, run old and new guards side by side with mismatch logging, migrate route-by-route, then delete the legacy path once confidence is established.
- **Signals integrate with `OnPush` only when read in a tracked context** (template expression, `computed()`, `effect()`) — a plain getter wrapping a signal read silently breaks automatic change detection and produces "works by accident" bugs.
- **Client-side authorization checks are always a UX/rendering layer.** No scenario in this chapter changes that: the server must independently enforce every rule, every time, regardless of what the client believes it knows.

Created By - Durgesh Singh